XML

Componentes XML

- XML se basa en el concepto de que un documento se compone de una serie de entidades (objetos)
  - Cada entidad contiene uno o más elementos
  - Cada elemento está caracterizado por cero o más atributos (propiedades)
- Las relaciones entre los elementos y la lista de sus posibles atributos se especifican en el Document Type Definition (DTD)
- Ejemplo:

```
<?xml version="1.0"?>
<!DOCTYPE mensaje [
<!ELEMENT mensaje (#PCDATA)>
]>
<!-- esto es un comentario -->
<mensaje>¡Hola, Mundo!</mensaje>
```

Tipos de datos, etiquetas, estructura básica

- Las anotaciones entre los símbolos < y > se denominan ***etiquetas*.** Hay etiquetas de inicio <nombre> y etiquetas de fin </nombre>
- **Los elementos** constan de la etiqueta de inicio, la etiqueta de fin y de todo aquello que este entre ambas

<mensaje>¡Hola, Mundo!</mensaje>

O por una etiqueta de elemento vacío

<imagen fichero="imagen.jpg"/>

<nombre>Sofía Fernandez</nombre> es un elemento

<nombre> es una etiqueta

Tipos de datos, etiquetas, estructura básica

- **Declaraciones**: son unas etiquetas "especiales" que dentro del documento XML sirve para declarar elementos necesarios para la interpretación de dicho XML
- `<!DOCTYPE peliculas SYSTEM "Peliculas.dtd" [<!NOTATION JPEG SYSTEM "Iexplore.exe">`
- **Atributos**: pueden contener **atributos** que almacenan características del elemento. Los atributos no le proporcionan esas características, sino que <u>únicamente las almacenan.</u>
- `<animal altura="25">Perro</animal>`

Tipos de datos, etiquetas, estructura básica

- Existen otros tipos de datos como las **entidades predefinidas** que permiten introducir caracteres especiales:

Entidad	Carácter
&	&
<	<
>	>
'	‘
"	“

Tipos de datos, etiquetas, estructura básica

- **Secciones CDATA** que permiten especificar datos, utilizando cualquier carácter, especial o no, sin que se interprete como marcado.

```
<parrafo>Lo siguiente es un ejemplo de HTML.
  <ejemplo>
            &lt;html>
                    &lt;head>&lt;title>Rock &amp;
            Roll&lt;/title>&lt;/head>
  </ejemplo>
  <ejemplo>
     <![CDATA[
       <html>
       <head><title>Rock & Roll</title></head>
       ]]>
  </ejemplo>
</parrafo>
```

- El único "carácter" no permitido en una cadena CDATA es "]]" que son el cierre de la sección CDATA. Todos los demás están permitidos.

Tipos de datos, etiquetas, estructura básica

- **Comentarios**: tienen el mismo formato que los comentarios de HTML. Es decir, en XML un comentario comienza por la cadena "<!--" y termina con "-->".

<!-- Esto es un comentario -->

- Se pueden incluir en cualquier parte excepto en la declaración de un elemento o dentro de otro comentario (como en HTML)

Ventajas de usar DTDs

- Cada organización, empresa, grupo, etc, puede definir su propio lenguaje para los documentos que suelen utilizar habitualmente
- Los nombres de las anotaciones son elegidos libremente, lo que facilita la comprensión de las mismas
- La aparición (en el futuro) de editores inteligentes
  - Ocultarán las anotaciones a los usuarios
  - Permitirá componer documentos correctos sintácticamente de forma sencilla

General

- Los documentos XML son sensibles a mayúsculas,
  - <FICHA> es una etiqueta diferente a <ficha>.
- Todos los espacios y retornos de carro se tienen en cuenta dentro de las etiquetas, en los elementos.
- Caracteres especiales reservados por formar parte de la sintaxis de XML: <, >, &, " , ‘ entidades <, >, &, " y '
- Los valores de los atributos de todas las etiquetas deben ir siempre entrecomillados.válidas las dobles comillas (") y la comilla simple (').

Documentos bien formados y válidos

- Un documento XML se dice que es bien formado si satisface un conjunto de reglas que permiten que una herramienta XML lo analice de forma correcta
- Un documento XML es válido si es bien formado y satisfacen las reglas que impone un DTD o un Schema

Documentos bien formados (I)

- Estructura jerárquica de elementos: los documentos XML deben seguir una estructura estrictamente jerárquica con lo que respecta a las etiquetas que delimitan sus elementos.
  - <td>HTML ****permite ****esto****</td>
  - <td>En XML la ****estructura ****es**** jerárquica****</td>
- Etiquetas vacías: XML permite elementos sin contenido, vacíos, pero la etiqueta debe cerrarse siempre de una de las siguientes maneras:

  <elemento-sin-contenido/>

  <elemento-sin-contenido></elemento-sin-contenido>

Documentos bien formados (II)

- Un solo elemento raíz: los documentos XML sólo permiten un elemento raíz, del que todos los demás sean parte.

```
<veterinario>
  <animal>
            <dueño></dueño>
      </animal>
      <especialidad>
            <titulo/>
  </especialidad>
</veterinario>
```

Documentos bien formados (III)

- Valores de atributos: los valores de atributos en XML, al contrario de HTML, siempre deben estar encerradas en comillas simples (') o dobles (").

  <enlace ruta="http://www.developer.com/">

- Tipo de letra y espacios en blanco:
  - el XML es sensible al tipo de letra utilizado (trata las mayúsculas y minúsculas como caracteres diferentes). Si un elemento de XML está definido como "ELEMENTO", no podemos usar "elemento", ni "Elemento", ni "eleMENto" para referirnos a él.

  - Existe un conjunto de caracteres denominados "espacios en blanco" que los procesadores XML tratan de forma diferente en el marcado XML. Estos caracteres son los "espacios" (Unicode/ASCII 32), tabuladores (Unicode/ASCII 9), retornos de carro (Unicode/ASCII 13) y los saltos de línea (Unicode/ASCII 10).La especificación XML 1.0 permite el uso de esos "espacios en blanco" para hacer más legible el código, y en general son ignorados por los procesadores XML.

Documentos bien formados (IV)

- Tipo de letra y espacios en blanco:
- Para que las aplicaciones preserven la representación de esos espacios en blanco en un elemento se utiliza el atributo **xml:space** que puede tomar los valores **preserve** o **default**, para conservar los espacios en blanco o aplicar el tratamiento por defecto, respectivamente.

```
<?xml version="1.0"?>
<textos>
    <cita> A quien madruga Dios le ayuda ?</cita>
      <poema xml:space="preserve">
        El monte es verde,
         el agua incolora
        y a mi no se me ocurre una rima
      </poema>
</textos>
```

- Hay que tener en cuenta que el espacio en blanco no está permitido antes de la declaración XML.

Documentos bien formados (V)

- Nomenclatura: al utilizar XML, es necesario asignar nombres a las estructuras, tipos de elementos, entidades, elementos particulares, etc. En XML los nombres tienen algunas características en común.
  - Un nombre [empieza] con una letra o uno o más signos de puntuación, y [continúa] con letras, dígitos, guiones, rayas, dos puntos o puntos, denominados de forma global como caracteres de nombre.
  - Los nombres que empiezan con la cadena 'xml' o 'Xml' o 'xML', etc, , se reservan para la estandarización de esta o de futuras versiones de esta especificación.
- Marcado y datos: las construcciones como etiquetas, referencias de entidad y declaraciones se denominan "marcas" (Son las que empiezan por < y acaban por >). Éstas son las partes del documento que el procesador XML espera entender. El resto del documento que se encuentra entre las marcas, son los datos que resultan entendibles por las personas

Documento Válido

- **XML válido** implica que el documento XML está bien formado y, además, sigue las reglas de un **DTD** (Document Type Definition).
- Los DTD explican qué tipos de etiquetas puede contener el archivo XML, qué hijos puede tener cuántos de cada tipo, qué tipo de contenido pueden tomar (texto, numérico, etc), qué atributos puede tener cada etiqueta, qué valores pueden tomar esos atributos y, también, la forma de relacionarse entre sí las etiquetas, es decir, su jerarquía.

```
<!DOCTYPE ficha [
  <!ELEMENT ficha (nombre+, apellido+, direccion+, foto?)>
  <!ELEMENT nombre (#PCDATA)>
  <!ATTLIST nombre sexo (masculino|femenino) #IMPLIED>
  <!ELEMENT apellido (#PCDATA)>
  <!ELEMENT direccion (#PCDATA)>
  <!ELEMENT foto EMPTY>
]>
```

Documento Válido

- Existe otra forma de establecer las reglas que debe cumplir un documento XML. Esa otra forma son los **Schema,** que son documentos similares a un DTD, es decir, definen qué elementos puede contener un documento XML, cómo están organizados, y que atributos y de qué tipo pueden tener sus elementos.
- Por ejemplo, un Schema nos permite definir el tipo del contenido de un elemento o de un atributo, y especificar si debe ser un número entero, o una cadena de texto, o una fecha, etc. Los DTD no nos permiten hacer estas cosas.

```
<Schema xmlns="urn:schemas-microsoft-com:xml-data"
        xmlns:dt="urn:schemas-microsoft-com:datatypes">
  <AttributeType name='id' dt:type='string' required='yes'/>
   <ElementType name='nombre' content='textOnly'/>
   <ElementType name='persona' content='mixed'>
        <attribute type='id'/>
        <element type='nombre'/>
  </ElementType>
</Schema>
```

Documentos bien formados y válidos

```
<!-- Este es un documento bien formado -->
<!-- Pero no es válido -->
<?xml version="1.0"?>
<mensaje>¡Hola, Mundo!</mensaje>

<!-- Este es un documento válido -->
<?xml version="1.0"?>
<!DOCTYPE mensaje [
<!ELEMENT mensaje (#PCDATA)>
]>
<mensaje>Hola, Mundo!</mensaje>
```

Componentes de un documento XML

- Los documentos en XML bien formados consisten en un prólogo, uno o más elementos y posiblemente información adicional
  - Instrucciones de procesamiento
  - Comentarios
- El prólogo contiene una declaración XML y una declaración del tipo de documento XML

Componentes de un documento XML

```
<!-- --------- Inicio del Prólogo ---------------------------->
<?xml version="1.0" standalone="yes"?> <!-- declaración XML -->

<!DOCTYPE mensaje [ <!-- declaración DOCTYPE -->
<!ELEMENT mensaje (#PCDATA)> <!-- declaración ELEMENT -->
]> <!-- final de declaración DOCTYPE -->
<!-- --------- Final del Prólogo --------------------------- -->

<!-- --------- Inicio de los elementos del documento -------- -->
<mensaje> Hola, Mundo! </mensaje>
<!-- --------- Final de los elementos del documento --------- -->
```

La declaración XML

- Es lo primero que un analizador XML se encuentra en un documento
- Declara cosas como
  - *version*: indica la versión de XML usada en el documento. Es obligatorio ponerlo, a no ser que sea un documento externo a otro que ya lo incluía.
  - *encoding*: la forma en que se ha codificado el documento. Se puede poner cualquiera, y depende del parser el entender o no la codificación. Por defecto es UTF-8, aunque podrían ponerse otras, como UTF-16, US-ASCII, ISO-8859-1, etc. No es obligatorio salvo que sea un documento externo a otro principal.
  - *standalone*: Si el documento está autocontenido, es decir, si el documento es independiente ("yes") o va acompañado de un DTD ("no"); en principio no hay porqué ponerlo, porque luego se indica el DTD si se necesita.

La declaración XML

```
<?xml version="1.0“ encoding=‘ISO-8859-1’
standalone=“yes”?>
<!DOCTYPE mensaje [
<!ELEMENT mensaje (#PCDATA)>
]>
<mensaje>
Hola, Mundo!
</mensaje>
```

DTD

DTD (I)

- La declaración de tipo de documento
  - comienza por la cadena "<!DOCTYPE"
- contiene, además
  - el tipo del *elemento raíz* del documento
- y declaraciones de anotaciones
  - especificadas de forma interna al documento,
  - especificadas mediante entidades externas,
  - o especificadas de ambas formas

DTD (II)

- Ejemplo de declaración interna

  <!DOCTYPE mensaje [<!ELEMENT mensaje (#PCDATA)>]>

- Ejemplos de declaración externa

  <!DOCTYPE mensaje SYSTEM “mensaje.dtd”>

  <!DOCTYPE mensaje SYSTEM http://www.abc.org/mens.dtd”>

  “mensaje” será el elemento raíz

DTD (III)

- Un DTD puede contener:
  - Declaraciones de elementos
  - Declaraciones de atributos
  - Declaraciones de entidades
  - Declaraciones de notación
  - Instrucciones de procesamiento
  - Comentarios

Declaraciones de elementos (I)

- Una declaración de elementos
  - Contienen el *tipo* de un elemento y su *modelo de contenido*
  - Comienzan por la etiqueta <!ELEMENT

<!ELEMENT nombre (modelo de contenido) >

Modelos de contenidos (I)

- #PCDATA : conocido como “Parser Character Data”, esto es, un dato que debe ser objeto de análisis. (entero, carácter, cadenas). Puede contener otros elementos

  <! ELEMENT miElemento (#PCDATA)>

- Otro elemento (o elementos) :

  <!ELEMENT clase (profesor)>

  - Que indica que clase sólo puede contener un solo elemento profesor.
  - En el caso que tenga varios subelementos como contenido, se habla de secuencia, que además determina el orden en que aparecen éstos, presentándolo siempre separados por comas; por ejemplo:

    <!ELEMENT clase (profesor,aula)>

  - Especifica que el elemento clase, debe contener un elemento profesor seguido de un elemento aula.

Modelos de contenidos (II)

- EMPTY: el elemento no tiene contenido y se llama elemento vacío escribiéndose de la forma :

  <!ELEMENT saltodepagina EMPTY>

- ANY: indica que el elemento puede tener cualquier contenido (aunque es muy conveniente que los elementos estén adecuadamente estructurados). Este valor admite la posibilidad de que elemento pueda tomar valores tan distintos como PCDATA, elementos o una combinación de PCDATA y elementos (incluso elementos vacíos);por ejemplo :

  <!ELEMENT batiburrillo ANY>

No se suele utilizar, ya que es conveniente estructurar adecuadamente nuestros documentos XML.

Modelos de contenidos (II)

- Mixed: indica que el elemento puede tener caracteres de tipo dato o una mezcla de caracteres y subelementos, pero al contrario que ANY sus contenidos deben estar debidamente especificados;por ejemplo :

```
<!ELEMENT enfasis ( #PCDATA)>
<!ELEMENT parrafo ( #PCDATA | enfasis )*>
```

  - Se usa el carácter barra (|) para separar las distintas opciones, con el significado de "o exclusivo", y los paréntesis para agrupaciones:

```
<!ELEMENT postre ( helado | pastel )>
<!ELEMENT postre ( sorbete, (helado | pastel))>
```

Modelos de contenidos (III)

- En la especificación de contenido se puede incorporar el identificador (también a una secuencia de ellos o a una opción) un símbolo de frecuencia, de acuerdo con la siguiente Tabla de significados :

Símbolo	*Descripción*
Ninguno	El elemento aparece sólo una vez.
Signo más (+)	El elemento aparece una o más veces.
Asterisco (*)	El elemento es opcional y puede aparecer cualquier número de veces.
Signo de interrogación (?)	El elemento es opcional y puede aparecer sólo una vez.

Modelos de contenidos (IV)

- Ejemplos

```
<!DOCTYPE receta [
<! ELEMENT receta ( plato, ingredientes, preparación)>
<!ELEMENT plato (#PCDATA)>
<!ELEMENT ingredientes (#PCDATA)>
<!ELEMENT preparación (#PCDATA)>
]>

<!DOCTYPE formato [
<!ELEMENT formato (#PCDATA | negrita | italica )*>
<!ELEMENT negrita (#PCDATA)>
<!ELEMENT italica ( #PCDATA)>
]>
```

Este DTD lo debes completar con lo que falta:

```
<!DOCTYPE formato [
<!ELEMENT aviso ( titulo?, (parrafo+, grafico)*)>
]>
```

Declaración de atributos

- Una declaración de atributo permite añadir información sencilla y desestructurada a los elementos de un documento; puesto que puede existir más de un atributo por elemento, se utiliza una lista para ello en la llamada declaración de lista de atributos (ATTLIST) cuya forma es :

  <!ATTLIST elemento nombre del atributo TYPE Palabra clave>

- Los atributos sobre un mismo elemento se pueden definir de forma separada o conjunta.

  <!ATTLIST persona peso CDATA #REQUIRED altura CDATA #REQUIRED>

Ejemplo

```
<!ELEMENT texto ( #PCDATA)>
<!ATTLIST texto idioma CDATA #REQUIRED>
<!ELEMENT mensaje (de, a, texto)>
<!ATTLIST mensaje prioridad (normal | urgente) “normal”>
<!ATTLIST mensaje origen CDATA “España”>
```

- Donde se indica que el atributo idioma pertenece al elemento texto pudiendo contener cualquier carácter, mientras que el atributo prioridad pertenece al elemento mensaje y puede tener el valor “normal” o “urgente”, siendo “normal” el valor por defecto si no se especificara este atributo en el documento.
- Respecto a la palabra clave #REQUIRED significa que no tiene valor por defecto, por lo que es obligatorio especificar este atributo. Si el atributo es opcional, se usa la clave #IMPLIED ya que a veces interesa que se pueda omitir un atributo, sin que se adopte automáticamente un valor por defecto.
- La tercera palabra clave #FIXED especifica que el valor del atributo es constante, y no puede cambiar a lo largo del documento. Al escribir :

  ```
  <!ATTLIST código postal #FIXED “46110”>
  ```

- La ultima posibilidad de palabra calve es un ***valor*** por defecto que va entrecomillado

Tipos de atributo (I)

- **CDATA** (datos de caracteres) que pueden tomar como valor cualquier secuencia o cadena de caracteres a excepción de los símbolos con significado especial : < , > , & , ' ,“ .
- **ID** indica que el atributo tiene un nombre definido, y con su uso se determina el valor aceptable para cada instancia del elemento al que se aplica. <!ELEMENT empleado (#PCDATA)>

  <!ATTLIST empleado nss ID #REQUIRED>

se trata de indicar que cada empleado tenga su nss y que dos empleados no pueden tener el mismo.

Tipos de atributo (II)

- **IDREF e IDREFS**. Mientras ID identifica a un elemento, IDREF apunta a un elemento con un atributo ID. IDREFS es una lista

Por ejemplo, para implementar un sistema de hipervínculos en un documento podemos utilizar :

```
<!ELEMENT enlace EMPTY>
<!ATTLIST enlace destino IDREF #REQUIRED>
```

Relacionado con los ER de bases de datos

https://www.mclibre.org/consultar/xml/lecciones/xml-dtd-resumen.html
http://www.mclibre.org/consultar/xml/lecciones/xml_dtd.html

Tipos de atributos (III)

- **Enumeraciones**. Corresponde a atributos que sólo pueden contener un valor de entre un número reducido de opciones, que aparecen en una lista proporcionada en la propia declaración (de aquí que se hable de enumeración) donde, además uno de estos valores se indica como valor por defecto. Por tanto, la elección del valor del atributo se limita a los valores que aparecen entre paréntesis y separados por el símbolo "|". Las enumeraciones son #REQUIRED siempre Así la expresión :

  <!ATTLIST teléfono lugar (oficina | movil | particular) "oficina">

Tipos de atributo (IV)

- **NMTOKEN** o **NMTOKENS**(Autenticaciones) son parecidos a las cadenas CDATA, aunque imponen restricciones sobre los valores de los atributos, de forma que sólo aceptan caracteres válidos para nombrar cosas (letras, números, puntos, guiones, subrayados y los dos puntos). NMTOKENS es una lista.

  <!ATTLIST pais población NMTOKEN #REQUIRED>

pais población =”500000” sería aceptado, pero no lo sería si el valor fuera escrito de la forma “5 000 000”debido a la presencia de espacios en blanco

Tipos de atributo (V)

- **xml:** El valor introducido será xml valido
- **ENTITY.** Permite especificar un atributo cuyo valor sea una entidad, a la que nos vamos a referir a continuación.
- **NOTATION**. Nos da información extra para la aplicación que procesara el xml

ENTIDADES (I)

En general entidad se refiere a un objeto usado para guardar información y por ello necesariamente cada documento tiene al menos la entidad del propio documento.

De la misma forma que los elementos describen la estructura lógica de un documento, las entidades mantienen la coherencia de la localización de los paquetes de bytes que constituyen la correspondiente estructura física, de forma que el concepto de entidad permite descomponer un documento en distintas unidades físicas contenidas en una o más unidades de almacenamiento.

ENTIDADES (II)

- Una entidad como colección de datos definidos y almacenados separadamente, tiene un nombre que sirve como referencia que "apunta" a la entidad. Las entidades se declaran mediante el uso de :

  <!ENTITY nombre "valor">

- donde nombre es el nombre de la entidad y valor es la cadena de sustitución correspondiente. Por ejemplo, si se define entidad y valor es la cadena de sustitución correspondiente. Por ejemplo, si se define una entidad llamada derechos para una publicación de un libro, se utilizaría :

  <!ENTITY derechos "Copyright 2002">

En XML

```
<texto><titulo>Los derechos de autor están basado en &derechos;</titulo></texto>
```

- En una entidad se llama texto de reemplazamiento al contenido de la entidad después de haber reemplazado los textos de caracteres de forma que, una vez reemplazada, pasa a ser una parte del documento y como tal es analizada.

ENTIDADES (III)

- **Entidades generales internas**
- Son abreviaturas definidas en la seccion de la DTD del documento. Son siempre entidades analizadas, es decir, una vez remplazada la referencia a la entidad por su contenido, pasa a ser parte del docuemento xml y es analizada por el procesador XML

```
<!DOCTYPE texto[
<!ENTITY ovni “Objeto Volante No Identificado””>]>
<texto><titulo>El día que vi un &ovni;</titulo></texto>
```

ENTIDADES (IV)

- **Entidad externa analizada** : difiere de la anterior en que su contenido no está dentro de la propia DTD, sino en cualquier otro sitio del sistema (otro archivo, un objeto de una base de datos, etc). Es analizada por el procesador XML. Se hace referencia a su contenido mediante una URI precedida de la palabra clave SYSTEM o PUBLIC, para dirigir al recurso externo nombrado por el URI que proporciona el valor de la entidad; por ejemplo :

```
<!ENTITY intro SYSTEM "http://www.miservidor.com/intro.xml">
```

ENTIDADES (V)

- **Entidades generales externas no analizadas**
- Si el contenido de una entidad es un archivo MEPG o una imagen GIF o un ejecutable, el procesador XML no debe intentar interpretarlo como si fuera texto XML.
- <!ENTITY logo SYSTEM “logo.gif”>

ENTIDADES (VI)

- **Entidad paramétrica interna** : solo pueden usarse dentro del la **DTD** no del documento **XML**, para poderlos escribir de forma abreviada; se distinguen por el uso de un nombre encabezado por (%). Su sintaxis es : <!ENTITY % nombre "valor">

Ejemplo:

```
<!DOCTYPE texto[
<!ENTITY %  elemento "<!ELEMENT mielemento (#PCDATA)>">
%elemento;
]>
```

Ejemplo:

```
<!DOCTYPE texto[
<!ENTITY % dimensiones "alto, ancho, largo">
<!ELEMENT pared (% dimensiones )>
<!ELEMENT techo (%dimensiones;)>
]>
```

ENTIDADES (VII)

- **Entidad paramétrica externa** : iguales que las anteriores, pero externas

Ejemplo:

```
<!DOCTYPE texto[
<!ENTITY % elemento SYSTEM “mielemento.ent”>
%elemento;
]>
```

ENTIDADES (VIII)

```
<!ENTITY %comun “talla (pequeña | media | grande ) 'media' color (rojo
  | azul | negro | blanco) 'blanco‘ precio CDATA #REQUIRED”>
<!ELEMENT calcetin (#PCDATA)>
<!ELEMENT panti (#PCDATA)>
<!ELEMENT medias (#PCDATA)>
```

Al escribir <!ATTLIST calcetin %comun> el analizador interpreta:

```
<!ATTLIST calcetin
talla (pequeña | media | grande ) 'media'
color (rojo | azul | negro | blanco) 'blanco'
precio CDATA #REQUIRED”>
```

Anotaciones (I)

La palabra inglesa notation tiene entre sus significados el de “notas” o “anotación”, dando la idea de **proporcionar información adicional**, cosa necesaria en un DTD para detallar el significado de los datos en un atributo o en una entidad para aplicaciones externas. (Indicar u tipo MIME por ejemplo)

Para poder utilizar esta información, antes hay que proceder a su declaración, cuya sintaxis general es de la forma :

```
<!NOTATION nombre SYSTEM “información de notación”>

<!NOTATION HTML SYSTEM “http://www.w3.org/Markup”>
<!NOTATION HTML PUBLIC “//W3C//DTD HTML 4.0 Transitional//EN”>
<!NOTATION gif SYSTEM “gif”>
```

Anotaciones (II)

Distingamos su uso en atributos y entidades.

- Para el caso de hacerlo dentro de atributos, las anotaciones se declaran añadiendo NOTATION a ATTLIST, lo que permite al autor del documento especificar en el momento de declarar el atributo que su valor se ajusta a una anotación dada.

```
<!ATTLIST imagen fecha NOTATION (ISODATE| EUROPEANDATE)
  #REQUIRED>
<!ATTLIST imagen fuente CDATA #REQUIRED tipo NOTATION (gif)
  #REQUIRED>
```

Anotaciones (III)

Para su uso dentro de una entidad, el esquema que se sigue es análogo al anterior y se lleva a cabo incorporando en la entidad externa la palabra clave NDATA. Veamos el uso de ello en el siguiente ejemplo : supóngase que se quiere incluir en la DTD un logo de una compañía del que se tienen dos versiones, una en GIF y otra en JPEG. Se empieza con las correspondientes declaraciones de la anotación :

```
<!NOTATION gif SYSTEM "gif">
<!NOTATION jpeg SYSTEM "jpeg">
<!ENTITY logogif
SYSTEM "imágenes/compañíalogo.
gif" NDATA gif>
```

Ejemplo

- En el DTD

```
<!NOTATION GIF SYSTEM "image/gif">
<!NOTATION JPG SYSTEM "image/jpeg">
<!NOTATION PNG SYSTEM "image/png">
<!ATTLIST mountain photo ENTITY #IMPLIED
photo_type NOTATION (GIF | JPG | PNG) #IMPLIED>
<!ENTITY mt_cook_1 "mt_cook1.jpg">
```

- En el XML

```
<mountains>
 <mountain photo="&mt_cook_1;" photo_type="JPG">
<name>Mount Cook</name>
</mountain>
<mountain>
<name>Cradle Mountain</name>
</mountain>
</mountains>
```

Secciones condicionales

Las DTDs tienen la posibilidad de incluir o ignorar declaraciones, así la palabra clave INCLUDE sirve para especificar las declaraciones que se incluyen e IGNORE las que son excluidas. El analizador incluye la declaración de un elemento nombre mediante la sintaxis :

```
<![ INCLUDE [
<!ELEMENT nombre (#PCDATA)>
]]>
```

Y excluye la declaración de un elemento mensaje mediante :

```
<![ IGNORE [
<!ELEMENT mensaje (#PCDATA)>
]]>
```

Secciones condicionales .Ejemplo

- El dtd mensaje.dtd

```
<!ENTITY % rechazar “IGNORE”>
<!ENTITY % aceptar “INCLUDE”>
<![ %aceptar; [
<¡ELEMENT mensaje ( aprobado, firma)>
]]>
<![ %rechazar; [
<!ELEMENT mensaje ( aprobado, razon, firma )>
]]>
<!ELEMENT aprobado EMPTY>
<!ATTLIST aprobado estado ( verdadero | falso ) “falso”>
<!ELEMENT razon ( #PCDATA )>
<!ELEMENT firma (#PCDATA)>
```

- En el XML

```
<?xml version="1.0"?>
<!DOCTYPE mensaje SYSTEM “mensaje.dtd" [
<!ENTITY % rechazar "IGNORE">
<!ENTITY % aceptar "INCLUDE">
]>
```

Resto del XML. Hmos definido un añadido apra usar las secciones condicionales

Ejemplo

Listin.DTD

```
<?xml encoding="UTF-8"?>
    <!ELEMENT listin (persona)+>
    <!ELEMENT persona (nombre, email*, relacion?)>
    <!ATTLIST persona id ID #REQUIRED>
    <!ATTLIST persona sexo (hombre | mujer) #IMPLIED>
    <!ELEMENT nombre (#PCDATA)>
    <!ELEMENT email (#PCDATA)>
    <!ELEMENT relacion EMPTY>
    <!ATTLIST relacion amigo-de IDREFS #IMPLIED
    enemigo-de IDREFS #IMPLID>
```

Listin.xml

```
<?xml version="1.0"?>
    <!DOCTYPE listin SYSTEM "listin.dtd">
    <listin>

    <persona sexo="hombre" id="ricky">
    <nombre>Ricky Martin</nombre>
    <email>ricky@puerto-rico.com</email>
    <relacion amigo-de="leatitia">
    </persona>

    <persona sexo="mujer" id="leatitia">
    <nombre>Leatitia Casta</nombre>
    <email>castal@micasa.com</email>
    </persona>

    </listin>
```

Espacios de nombres

Espacios de nombres (I)

- Un **espacio de nombres XML (namespace XML)** como un conjunto de nombres, los cuales se identifican por una referencia URL
- Así, se evitan conflictos derivados del uso de etiquetas o atributos con el mismo nombre pero que pertenezcan a dos elementos distintos

```
<cliente>
<nombre>Juan Pérez</nombre>
<mascota>
  <nombre>Scooby</nombre>
</mascota>
</cliente>
```

Espacios de nombres (II)

- En este escenario habría un conflicto entre las etiquetas <nombre> del cliente y de la mascota. Sería necesario hacer una distinción entre ellas, saber a qué corresponde cada una, si al cliente o a la mascota.
- Para ello, se puede asignar un espacio de nombres propio a cada grupo de datos, los del cliente y los de la mascota.
- Para declarar un espacio de nombres que afecte a una etiqueta (elemento) se usará el atributo **xmlns**. Dicho atributo xmlns puede ir seguido de un identificador para el espacio de nombres y separados por ":". Por ejemplo:

```
<nombrens:elemento xmlns:nombrens="http://www.uriprueba.com/ns">
    <!— contenido de La etiqueta "elemento" →
</ nombrens:elemento/>
```

Espacio de nombres (III)

- Si se utiliza un nombre, ese nombre se debe poner como prefijo de los elementos que pertenezcan a ese espacio de nombres, irá seguido de ":" y a continuación el nombre del elemento. El prefijo también irá en las etiquetas de cierre:

```
<nombrens:elemento xmlns:nombrens="http://www.uriprueba.com/ns">
        <nombrens:subelemento1>Dato subelemento 1</ nombrens:subelemento1>
        <nombrens:subelemento2>Dato subelemento 2</ nombrens:subelemento2>
</nombrens:elemento>
```

Espacio de nombres (IV)

En este caso se utiliza un prefijo como nombre del espacio de nombres:

```
<cli:cliente xmlns:cli='http://www.prueba.com/cliente'>
    <cli:nombre>Juan Pérez</cli:nombre>
<pet:mascota xmlns:pet='http://www.prueba.com/pet'>
    <pet:nombre>Scooby</pet:nombre>
</pet:mascota>
</cli:cliente>
```

- En este caso se identificaría que la etiqueta <cli:nombre> pertenece al espacio de nombres **cli**, que está asociado a la URL http://www.prueba.com/cliente y que se asigna a todos los elementos hijos del elemento <cli:cliente> y a él mismo, es decir, en este caso al cliente
- Del mismo modo, el espacio de nombres **pet** se asocia a los elementos correspondientes a la mascota, asociándolo a la URL http://www.prueba.com/pet y diferenciando por tanto su etiqueta <pet:nombre> que indicará que ese es el nombre de la mascota.
- El atributo **xmlns** se usa solo para declarar un namespace. , no para ligarlo/asociarlo a un namespace. Así en el ejemplo:

  <elemento xmlns:pre="**http://www.prueba.com**" />
- Lo que se asocia al namespace "http://www.prueba.com." es el prefijo “pre”.

Espacios de nombres (V)

- Lo primero para usar un namespace es asignarle un prefijo, el cual se usará donde se requiera. Incorrecto: <http://www.libreria.com:Libro />

  Correcto: <lib:Libro xmlns:lib="http://www.libreria.com" />

- Por ejemplo, en el siguiente código los elementos titulo y autor están asociados con el namespace http://www.libreria.com:

```
<?xml version="1.0"?>
<Libro xmlns:lib="http://www.libreria.com">
    <lib:Titulo>Sherlock Holmes</lib:Titulo>
    <lib:Autor>Arthur Conan Doyle</lib:Autor>
</Libro>
```

- Para evitar sobrecargar el código con prefijos se puede asignar un namespace por defecto, que afectará a todos los elementos hijos que no utilicen explícitamente otro prefijo.

  <elemento **xmlns="http://www.prueba.com"** />

- Los namespace por defecto no se aplican a los atributos, por tanto cualquier namespace aplicado a un atributo tiene que estar asociado a un prefijo.

  <elemento **xmlns:pre="http://www.prueba.com"** pre:atributo="valor" />

- Los namespaces se puede sobrescribir o desasignar, tanto los namespaces por defecto como los que están asociados a un prefijo

  Por defecto <elemento **xmlns=""** />

  Con prefijo <elemento **xmlns:pre=""**>

- Una vez eliminada la declaración de un namespace, dicho namespace no se puede utilizar en el ámbito de la etiqueta donde se haya eliminado la declaración, pero se puede volver a declarar en cualquier punto

Espacios de nombres (VI)

- El ámbito de la declaración de un namespace comienza en el elemento en el que se declara y se aplica a todos sus hijos, excepto si es sobrescrito por otro namespace con el mismo prefijo.
- Tanto los namespaces con prefijo como los namespaces “por defecto” se puede sobrescribir.
- Se puede eliminar la declaración tanto de los namespaces con prefijo como de los namespaces “por defecto”.
- Los namespaces “por defecto” no se aplican a los atributos directamente
- Los namespaces “por defecto” existen únicamente si se han decalrado explicitamente; es incorrecto utilizar el término “namespace por defecto” para un documento en el que no se haya declarado, aunque no haya ningún otro namespace con prefijo declarado